



Catch the migration that locks production — before you ship it.

A PostgreSQL migration gate that measures real locks and uses a custom **IBM Bob** mode with parallel subagents to rewrite dangerous migrations into verified zero-downtime rollouts.

Migration risk is invisible in a diff

INNOCENT-LOOKING LINE	WHAT POSTGRESQL DOES
CREATE INDEX ... ON orders (customer_id)	ShareLock for the whole build → all writes blocked
ALTER COLUMN total TYPE numeric(12,2)	AccessExclusiveLock + full table rewrite
ADD COLUMN ... NOT NULL DEFAULT now()	volatile default → full table rewrite
RENAME COLUMN email TO email_address	instant — but every running app instance breaks
...no SET lock_timeout	queues behind one slow query, then blocks everything

Reviewers need lock semantics by heart. Misses cause outages; the safe rewrite costs a senior engineer hours.

THE SOLUTION

Detect → Prove → Impact → Rewrite with Bob → Re-verify

01

Detect

12 deterministic rules,
incl. renames still
referenced by app
code

02

Prove

Replay in embedded
Postgres (PGLite); read
pg_locks, detect
rewrites via
relfilenode

03

Impact

Table sizes → seconds
of lock, writes
blocked

04

Rewrite · IBM Bob

Custom mode + skill;
1 subagent per
migration, in parallel

05

Gate

Same engine re-
checks Bob's output;
CLI blocks CI

The AI is verified, not trusted: a deterministic engine with measured locks judges; Bob does the judgement-heavy surgery.

EVIDENCE, NOT GUESSES

Every finding carries the lock Postgres actually took

001_baseline.sql

safe +

002_orders_customer_index.sql LS001 LS010

100 -

```
1 -- Speed up "my orders" page
2 CREATE INDEX idx_orders_customer_id ON orders (customer_id);
3
```

CRITICAL LS001 line 2

CREATE INDEX without CONCURRENTLY on table "orders"

Acquires a ShareLock for the entire duration of the index build, blocking writes. On large tables this can take minutes.

MEASURED IN PGLITE

orders **ShareLock** · blocks writes

idx_orders_customer_id **AccessExclusiveLock** · blocks reads+writes

Est. lock held 28 s (scan) · ~9,800 writes blocked

► Safe pattern

MEDIUM LS010 line 2

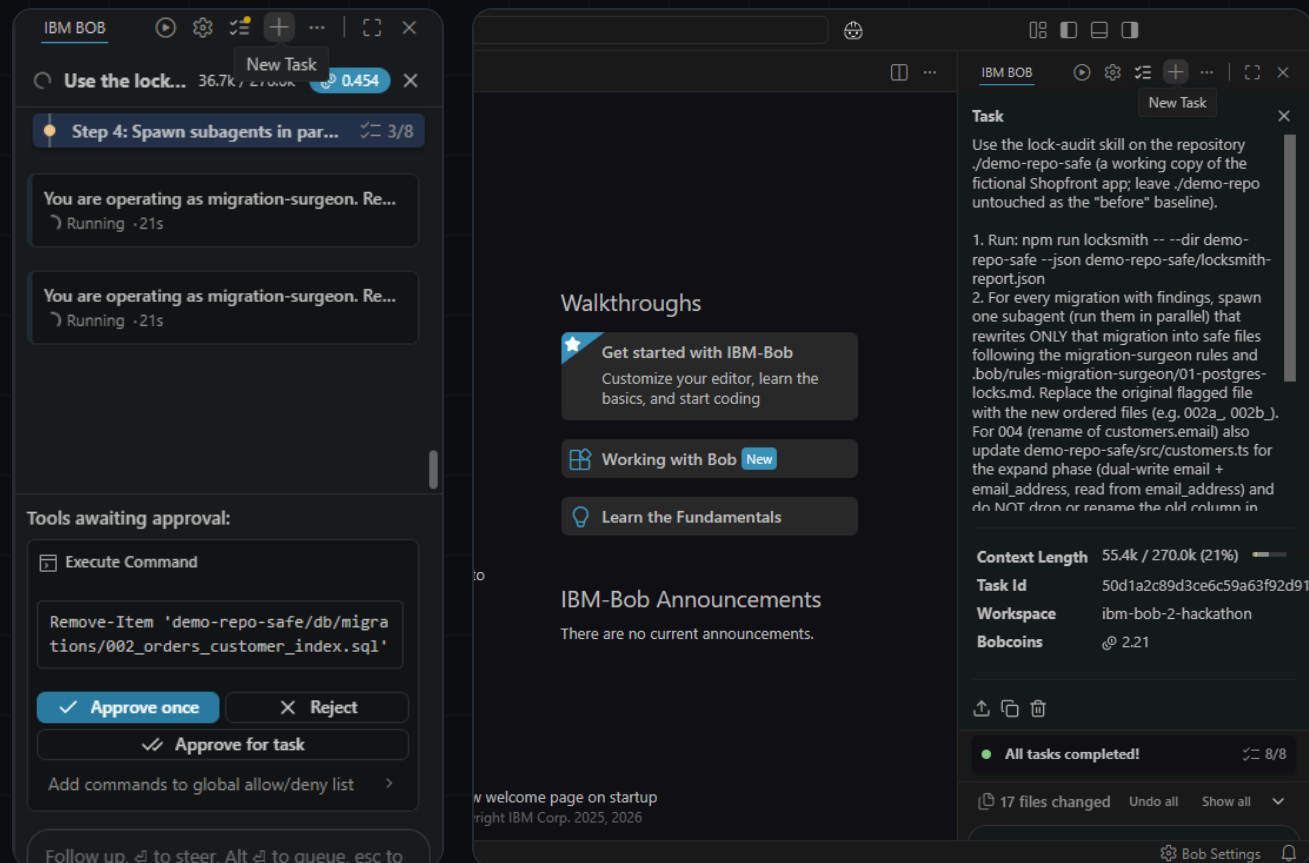
Migration acquires a heavy lock but has no SET lock_timeout

Without lock_timeout, a long-running query ahead of your DDL will block it, and then every subsequent query queues behind it (lock queue pile-up), causing an outage.

MEASURED IN PGLITE

orders **ShareLock** · blocks writes

Migration Surgeon: a Bob mode that ships in the repo



- `.bob/custom_modes.yaml` — role, fileRegex-restricted edits
- `.bob/rules-migration-surgeon/` — lock reference Bob reads
- `.bob/skills/lock-audit` — run gate → **6 parallel subagents** → merge → re-run gate
- Bob withdrew its own column-drop when it saw the app still used the column

RESULT ON THE DEMO REPOSITORY (FICTIONAL, 42M-ROW ORDERS TABLE)

Gate **FAIL** → **PASS** after Bob's rewrite

AS WRITTEN

100

risk · **FAIL** · 14 findings (4 critical) · longest lock $\approx$ 4.7 min

AFTER IBM BOB

0

risk · **PASS** · 0 findings · 6 migrations → 13 ordered safe files + dual-write app code

Verified by the same engine and lock probe. 61 automated tests. Try any migration at </analyze>.

HOW WE USED IBM BOB TO BUILD IT

6 Bob IDE tasks · 13.07 of 40 Bobcoins

#	TASK	BOB FEATURES
1	Architecture + contracts from the spec	Agent mode · document understanding
2	Rule engine LS001–LS012 + 53 tests	Agent mode · todo list · test loop
3	PGlite lock probe + fix of a reviewed defect	parallel commands · debugging
4	Migration Surgeon mode, rules, skill	custom mode · skill
5	Rewrite the release	custom mode · 6 parallel subagents
6	Engine false positive + API security review	debugging · review

Every task: summary screenshot in bob_sessions/, independent re-run of tests, accept/reject logged.

BUSINESS VALUE & NEXT STEPS

Turn "does this lock orders? for how long?" into a CI check — and hours of rewriting into a verified Bob task.

Next: GitHub Action · Prisma / Rails / Flyway adapters · real table sizes from `pg_stat_user_tables` · MySQL online-DDL rules · watsonx.ai rollout runbooks.